7CS039 Lab 3 Correlation & Regression

1 Objectives

By the end of this session we will be able to

- Perform correlation and regression analysis using `R`.
- Interpret results from correlation and regression.
- Perform residual analysis using `R` to check the assumptions of regression.

2 Bodyfat

We are going to be using the body fat data set today, You can download it from canvas. Then load it into `R` as usual and preview the data.

```
> head(Bodyfat)
# A tibble: 6 x 15
  Density bodyfat   Age Weight Height  Neck Chest Abdomen   Hip Thigh  Knee Ankle Biceps Forearm Wrist
    <dbl>   <dbl> <dbl>  <dbl>  <dbl> <dbl> <dbl>   <dbl> <dbl> <dbl> <dbl> <dbl>  <dbl>   <dbl> <dbl>
1    1.07    12.3    23   154.   67.8  36.2  93.1    85.2  94.5  59    37.3  21.9   32      27.4  17.1
2    1.09     6.1    22   173.   72.2  38.5  93.6    83    98.7  58.7  37.3  23.4   30.5    28.9  18.2
3    1.04    25.3    22   154    66.2  34    95.8    87.9  99.2  59.6  38.9  24     28.8    25.2  16.6
4    1.08    10.4    26   185.   72.2  37.4 102.     86.4 101.   60.1  37.3  22.8   32.4    29.4  18.2
5    1.03    28.7    24   184.   71.2  34.4  97.3   100   102.   63.2  42.2  24     32.2    27.7  17.7
6    1.05    20.9    24   210.   74.8  39   104.     94.4 108.   66    42    25.6   35.7    30.6  18.8
>
```

We have 252 observations across 15 categories. The variables in the data set are (from left to right):

- Density determined from underwater weighing
- Percent body fat from Siri's (1956) equation
- Age (years)
- Weight (lbs)
- Height (inches)
- Neck circumference (cm)
- Chest circumference (cm)
- Abdomen 2 circumference (cm)

- Hip circumference (cm)
- Thigh circumference (cm)
- Knee circumference (cm)
- Ankle circumference (cm)
- Biceps (extended) circumference (cm)
- Forearm circumference (cm)
- Wrist circumference (cm)

Now let's attach the data set and compute some correlation coefficients to get a feeling for the data. We'll use the `cor(x,y, ...)` function although we could use the `lm()` function as we did earlier in the week.

```
> attach(Bodyfat)
> cor(Age, bodyfat, method="pearson")
```

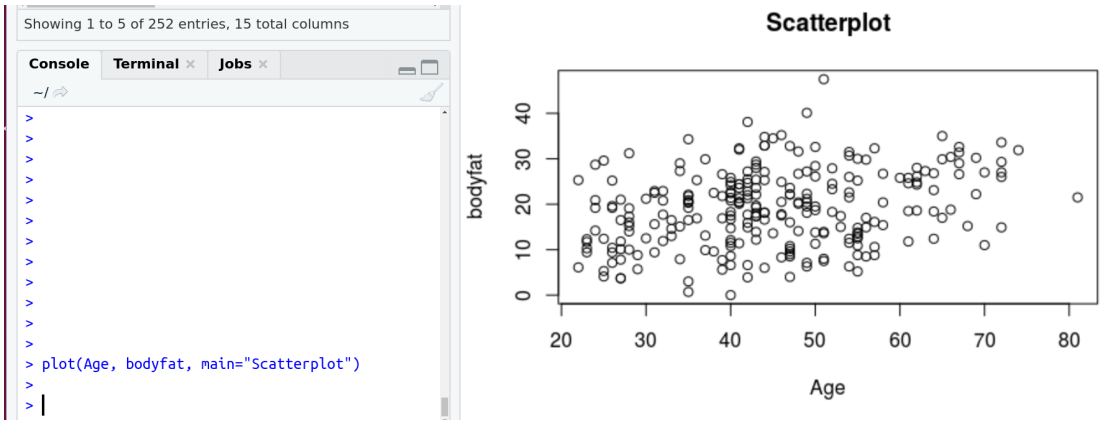
```
> attach(Bodyfat)
The following objects are masked from Bodyfat (pos = 3):

    Abdomen, Age, Ankle, Biceps, bodyfat, Chest, Density, Forearm, Height, Hip, Knee, Neck, Thigh, Weight, Wrist

> cor(Age, bodyfat, method="pearson")
[1] 0.2914584
>
```

So we have $r = 0.291$. What does this mean? Let's construct a scatterplot of age ($x - axis$) vs bodyfat ($y - axis$).

```
> plot(Age, bodyfat)
```



Now let's build our regression model (note that it's $y \sim x$).

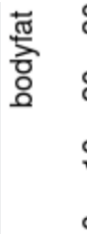
```
> my_model=lm(formula = bodyfat~Age)
```

```
> plot(Age, bodyfat, main="Scatterplot")
>
> my_model=lm(formula = bodyfat~Age)
> my_model

Call:
lm(formula = bodyfat ~ Age)

Coefficients:
(Intercept)          Age
    10.4633       0.1936

>
```



So the regression equation for our model is

$$y = 0.19x + 10.5$$

where $y = \text{bodyfat}, x = \text{Age}$. Next we do

```
> summary(my_model)
```

```
> summary(my_model)

Call:
lm(formula = bodyfat ~ Age)

Residuals:
     Min       1Q   Median       3Q      Max
-18.2053  -6.1513   0.2979   5.3665  27.1656

Coefficients:
            Estimate Std. Error t value
(Intercept) 10.46326    1.87278   5.587
Age          0.19355    0.04018   4.818
            Pr(>|t|)
(Intercept) 6.02e-08 ***
Age         2.52e-06 ***
---
Signif. codes:
0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 8.021 on 250 degrees of freedom
Multiple R-squared:  0.08495,   Adjusted R-squared:  0.08129
F-statistic: 23.21 on 1 and 250 DF,  p-value: 2.522e-06
```

and we see that $r = \sqrt{0.084985} \simeq 0.291$ as before. How do we interpret this value for r?

Activity

Repeat the above analysis for any choice of two variables from the dataset.

2.1 Understanding r

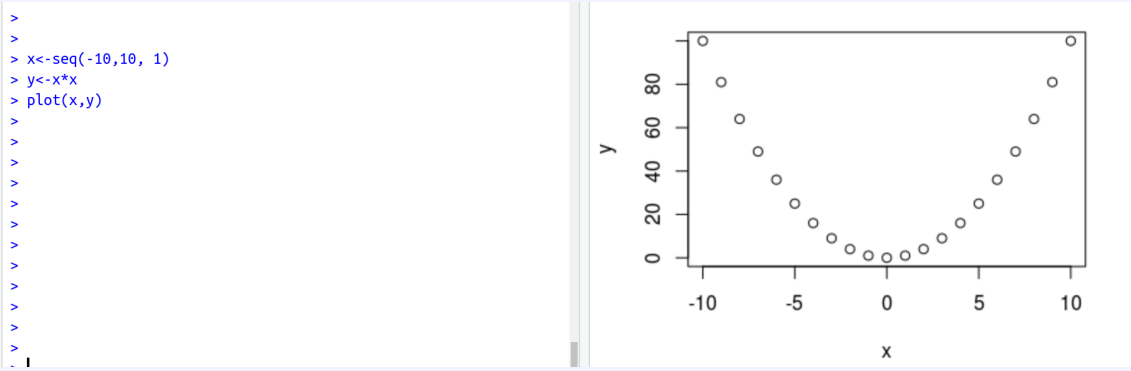
Correlation is very useful but it is also one of the most misused statistics in all of science. It isn't always possible to summarize the relationship between two variable with a single

number. If we obtain a correlation coefficient of 1 or -1 we have a good idea of what it means, but how do we interpret a correlation coefficient of 0.291? Let's look at some extreme cases.

Example

Consider $y = x^2$ and let's generate some points on this curve in steps of size 1 as follows

```
> x<-seq(-10,10, 1)
> y<-x*x
> plot(x,y)
```



We know that there is a perfect quadratic relationship between x and y and that this relationship is described by $y = x^2$. But let's pretend we don't know that and let's ask for the correlation coefficient.

```
> x
> y
> cor(x,y)
```

```
> x
 [1] -10  -9  -8  -7  -6  -5  -4  -3  -2  -1   0   1   2   3   4
[16]   5   6   7   8   9  10
> y
 [1] 100  81  64  49  36  25  16   9   4   1   0   1   4   9  16
[16]  25  36  49  64  81 100
> cor(x,y)
[1] 0
>
```

So $r = 0$ even though we know that there is a very strong relationship between x and y. This example illustrates the importance of remembering that we are only looking at "linear" relationships. Just because $r = 0$ does not mean that there is no relationship.

Activity

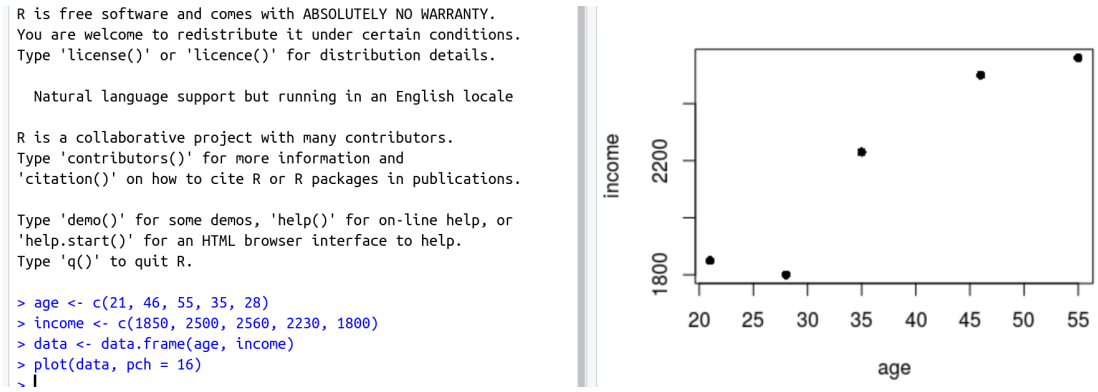
Repeat the analysis from the above example for $y = x^3, y = x^4, y = x^5, \ldots$. Comment on the results.

3 Underfitting & Overfitting

One of the problems with regression and correlation is over/under fitting. We'll now explore this. The material that follows is taken from Prof. Dr. Holger K. von Jouanne-Diedrich's blog https://blog.ephorie.de/ which I highly recommend.

Data Science is all about building good models, so let us start by building a very simple model: we want to predict monthly income from age. We'll build a data set (data frame) from scratch for this example.

```
> age <- c(21, 46, 55, 35, 28)
> income <- c(1850, 2500, 2560, 2230, 1800)
> data <- data.frame(age, income)
> plot(data, pch = 16)
```



Now we can use the `cor()` function to produce a correlation "matrix"

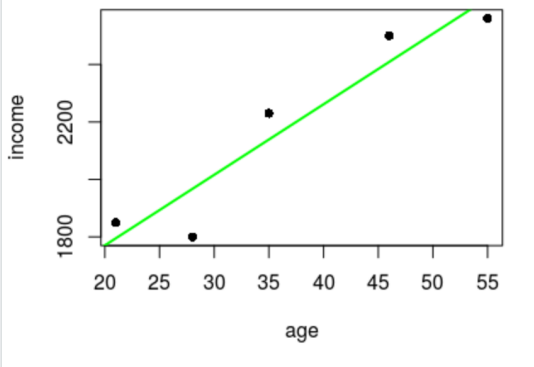
```
> cor(data)
```

```
>
>
> cor(data)
             age    income
age    1.0000000 0.9464183
income 0.9464183 1.0000000
>
```

Now let's build our linear model as usual

```
> LinReg <- lm(income ~ age)
> abline(LinReg, col = "green", lwd = 2)
```

```
>
>
>
>
>
>
>
>
>
>
>
>
> cor(data)
             age    income
age    1.0000000 0.9464183
income 0.9464183 1.0000000
> LinReg <- lm(income ~ age)
> abline(LinReg, col = "green", lwd = 2)
>
```



As usual we view the summary our model via:

```
> LinReg
```

```
> LinReg

Call:
lm(formula = income ~ age)

Coefficients:
(Intercept)          age
    1279.37        24.56
```

```
> summary(LinReg)
```

```
> summary(LinReg)

Call:
lm(formula = income ~ age)

Residuals:
      1       2       3       4       5
  54.92   90.98  -70.04   91.12 -166.98

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 1279.367    188.510   6.787  0.00654 **
age           24.558      4.838   5.076  0.01477 *
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 132.1 on 3 degrees of freedom
Multiple R-squared:  0.8957,    Adjusted R-squared:  0.8609
F-statistic: 25.77 on 1 and 3 DF,  p-value: 0.01477
```

The general line equation is $y = mx + b$, where m is the slope/gradient (in this case 24.558) and b the intercept (here 1279.367). If we now want to predict the income of a

40-year old we just have to calculate

$$1279.37 + 40 \cdot 24.56 = 2261.67.$$

To do the same calculation in R we can conveniently use the predict function which does this calculation for us automatically. We can also use vectors as input and get several predictions at once. Important is that we use the independent variable with its name as a data frame, otherwise R will throw an error.

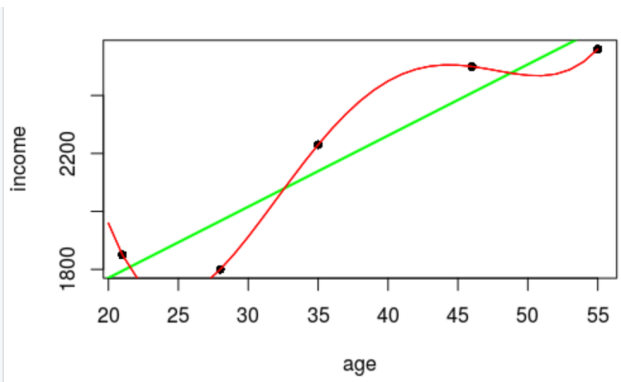
```
> predict(LinReg, data.frame(age = 40))
> pred_LinReg <- predict(LinReg, data.frame(age = seq(from = 0, to = 80, by = 5)))
> names(pred_LinReg) <- seq(0, 80, 5)
> pred_LinReg
```

```
> predict(LinReg, data.frame(age = 40))
       1
2261.673
> pred_LinReg <- predict(LinReg, data.frame(age = seq(from = 0, to = 80, by = 5)))
> names(pred_LinReg) <- seq(0, 80, 5)
> pred_LinReg
       0        5       10       15       20       25
1279.367 1402.155 1524.944 1647.732 1770.520 1893.308
      30       35       40       45       50       55
2016.097 2138.885 2261.673 2384.461 2507.249 2630.038
      60       65       70       75       80
2752.826 2875.614 2998.402 3121.190 3243.979
>
```

So far so good, this is what we have been doing already and there is nothing new here. Wouldn't it (would it?) be nice to actually have a model that is even more accurate, a model where the line actually goes through all the data points so that all the available information is being used. Let us do that with so-called "polynomial regression" (in red):

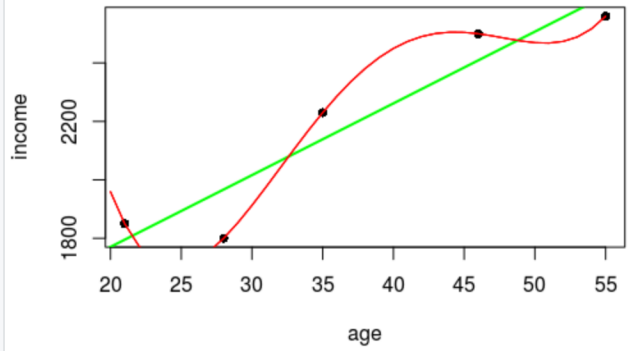
```
> PolyReg <- lm(income ~ poly(age, 4, raw = TRUE))
lines(c(20:55), predict(PolyReg, data.frame(age = c(20:55))), col = "red")
```

```
>
>
>
>
>
>
> PolyReg <- lm(income ~ poly(age, 4, raw = TRUE))
> lines(c(20:55), predict(PolyReg, data.frame(age = c
(20:55))), col = "red")
>
>
>
>
>
>
>
>
>
```




```
> pred_PolyReg <- predict(PolyReg, data.frame(age = seq(0, 80, 5)))
names(pred_PolyReg) <- seq(0, 80, 5)
> pred_PolyReg
```

```
>
>
>
>
>
>
> pred_PolyReg <- predict(PolyReg, data.frame(age = s
eq(0, 80, 5)))
> names(pred_PolyReg) <- seq(0, 80, 5)
> pred_PolyReg
         0          5         10         15         20
19527.182 11103.509  5954.530  3164.488  1959.098
        25         30         35         40         45
 1705.541  1912.468  2230.000  2449.726  2504.705
        50         55         60         65         70
 2469.464  2560.000  3133.778  4689.732  7868.267
        75         80
13451.255 22362.037
>
```



Is this a better model? Why? Why not? There is no good reason why income should go up the younger or older you are apart from obvious cases), and there is no good reason why there should be a bump between 50 and 55 for example. In other words, this model takes the noise as legitimate input which actually makes it worse! To have a good model you always need some rigidity to generalize the data. This illustration is a good example of one of the big problems of data science & machine learning: overfitting the data! The other extreme would be underfitting, which is included in the following plot as a blue line:

```
> abline(h = mean(income), col = "blue")

> title(main = "All models are wrong, some are useful")
legend("bottomright", c("Mean: Underfitting", "Linear fit",
"Polynomial Reg.: Overfitting"),
col = c("blue", "green", "red"), lwd = 3)

> mean(income)
```

We just took the mean value of income and claimed that no matter the age everybody would earn about 2200. This also obviously doesn't make the cut. In practice and in all data science projects you always have to look for a model that hits the sweet spot between over and underfitting. This is as much an art as a science, although there are certain best practices like dividing the data into a training and a test set, cross-validation, etc.

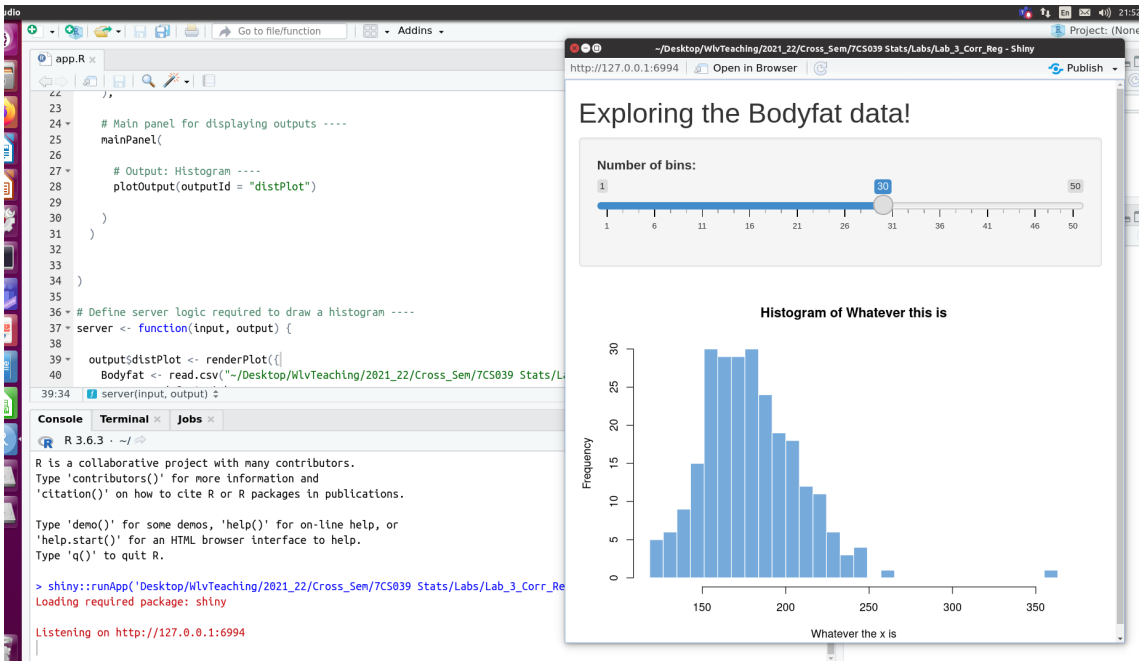
4 Building your own App using Shiny

For the remainder of this session we are going to look at how we can build our own Apps for probing data using the `shiny` package. The `R Shiny` framework is a package from `RStudio` that makes it really easy to build interactive web applications with `R`. `Shiny` allows you to create highly effective data reports and visualizations where the user can explore a data set. This is very important for you because as part of your MSc project you need to create an "artefact" as well as a report.

`Shiny` applications are divided into two parts: the User Interface (UI) and the Server. The UI is responsible for the app presentation (how it looks), while the server is responsible for the app logic (i.e. it contains `R` commands). let's start by installing `Shiny`.

```
> install.packages("shiny")
> library("shiny")
```

Now we'll use the sample App that's on canvas. We'll start by creating a folder on our desktop called "MyApp" and then we'll download the file `app.R` from canvas into that folder. The sample App allows us to create a histogram using the `Bodyfat` dataset.



Now let's look at the code for the App in more detail to get a feel for what's actually happening. We have a UI part and a Server and then the App is created by the line

```
> shinyApp(ui = ui, server = server)
```

Activity

Change the code so that your App displays a histogram for one of the other variables.

That's all we can do today. I can't make you an expert in `Shiny` - only you can do that. A good place to start is

https://mastering-shiny.org/basic-app.html

If you have your own dataset e.g. from your job or from a survey then you can produce a professional product very quickly using `shiny`. Imagine how impressed your boss will be!